Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim **Ön Bilgi:** Hf.3'ün `querySelector/classList` ile iki ayrı yerde elle güncelleme sorunu hatırlanır; YG I/II'nin `const` disiplini `props`'a köprülenir; `tipler.ts/veri.ts` Hf.3'ten aynen taşındı.

» Ünite 1 — Neden Bileşen? React'e Giriş ve TSX

Bildirimsel vs Zorunlu Programlama

Zorunlu yaklaşımda DOM'u nasıl güncelleyeceğinizi adım adım siz yazarsınız (`querySelector` → bul → `classList.toggle`); bildirimsel yaklaşımda yalnızca durum ne ise görünüm ne olmalı tanımlarsınız — React günceller.

React ve Sanal DOM

React her güncellemede bir taslak ağaç (sanal DOM) hesaplar, öncekiyle karşılaştırır (uzlaştırma) ve yalnızca farkı gerçek DOM'a uygular.

Vite `react-ts` Kurulumu

`npm create vite@latest -- --template react-ts` — `vanilla-ts`'e çok benzer (`index.html`, `tsconfig.json`) ama artık `react/react-dom` bağımlılıkları ve `.tsx` dosyaları içerir.

TSX: İfade Gömme

JSX/TSX, JavaScript fonksiyon çağrılarına derlenen, HTML'e benzeyen bir sözdizimi. Süslü parantez içine yalnızca bir değer üreten ifade yazılır — `if/for` gibi bir deyim yazılamaz; `class` yerine `className` kullanılır.

Tek Kök Öğe ve Fragment

Bir bileşen tek bir kök öğe döndürmelidir; birden fazla kardeş öğe gerekiyorsa görünmez bir sarmalayıcı olan Fragment (`<>...</>`) kullanılır.

Ortak Hata / Tuzak

`<h1>`/`<p>`'yi doğrudan kardeş olarak döndürmek `tsc` hatası verir — bir bileşen tek kök öğe döndürmelidir; çözüm ortak bir `<div>` ya da Fragment (`<>...</>`).

» Ünite 2 — props ve Tiplenmiş Bileşen

Fonksiyon Bileşeni

JSX döndüren sıradan bir TypeScript fonksiyonu; büyük harfle başlayan adlandırma zorunlu — React küçük harfle başlayan adları HTML etiketi sanır.

`props` (Özellikler)

Bir bileşene dışarıdan veri geçirmenin tek yolu; fonksiyona parametre geçirmeye benzer ama salt okunur kabul edilir — bileşen kendisine gelen `props`'u doğrudan değiştirmez.

`interface XProps` ile Tipleme

Her bileşenin aldığı `props`'un şeklini bir `interface` tanımlar; yanlış tipte ya da eksik bir prop geçilince `tsc` hata verir.

`children` (Çocuk Öğeler)

Bileşenin açılış/kapanış etiketleri arasına yazılan JSX, `props.children` olarak otomatik gelir — bileşeni bir sarmalayıcı (*wrapper*) olarak kullanmayı sağlar.

Bileşen Ağacı ve Tek Yönlü Veri Akışı

`props` yalnızca yukarıdan aşağıya akar — bir alt bileşen kendi verisini üst bileşene doğrudan gönderemez (Hf.5'in "durum kaldırma" konusu).

Ortak Hata / Tuzak

`etkinlik.baslik = 'Yeni Başlık'` ataması `tsc`/React tarafından otomatik engellenmez ama tek yönlü veri akışını bozar — değişiklik üst bileşene bir olay/geri çağırım ile bildirilir (Hf.5).

» Ünite 3 — state ve Olaylar

`useState` Hook'u

`const [durum, setDurum] = useState<T>(baslangic)` — bir bileşenin render'lar arasında hatırlanan kendi verisini tutmasını sağlayan React "hook"u; iki eleman döner: mevcut değer ve onu değiştirmenin tek yolu.

Değişmezlik İlkesi

Bir state değerini doğrudan değiştirmek (mutasyon) yerine her zaman yeni bir değer üretip `setX(...)`'e verilir — "eski değeri değiştirmeyin, yenisini bildirin" kuralını derleyici değil React'in kendisi koyar.

Olay İşleyicileri

`onClick`, `onSubmit` gibi JSX olay öznitelikleri; React bunları normal DOM olaylarının üstüne tipli bir katman olarak sunar (`(e: React.MouseEvent) => ...`).

Durumun Bileşen İçinde Yaşaması

Bir `useState` çağrısı yalnızca tanımlandığı bileşene aittir — başka hiçbir bileşen bu değeri doğrudan göremez/değiştiremez.

Ortak Hata / Tuzak

`useState`'in döndürdüğü ilk değer `const` bağlamlıdır — `durum = 'secili';` doğrudan ataması derleyici hatası verir; state'i değiştirmenin tek yolu `setDurum(...)` çağırmaktır, doğrudan atama React'e hiçbir şey söylemez, ekran değişmez.

Yerel durum ve tıklama (`koltuk_state_ve_tiklama` etiketi):

```
// Koltuğun kendi durumu, koltuk.durum başlangıç değeriyle yerel bir
// useState'te tutulur; App ya da KoltukPlani bu değeri GÖRMEZ.
const [durum, setDurum] = useState<KoltukDurumu>(koltuk.durum);

function tikla() {
  if (durum === 'dolu') return;
  setDurum((onceki) => (onceki === 'bos' ? 'secili' : 'bos'));
}
```

» Ünite 4 — Liste ve Koşullu Render

`map` ile Bileşen Listesi

Bir diziden (`Etkinlik[]`, `Koltuk[]`) `.map(...)` ile bir JSX öğe dizisi üretilir — Hf.3'teki `forEach` + `createElement` döngüsünün bildirimsel karşılığı.

`key` Neden Gerekli

Listedeki her öğeye veriden gelen, kararlı ve benzersiz bir kimlik (`etkinlik.id`) vermek, React'in uzlaştırmada "bu hangi öğeydi" sorusunu doğru cevaplamasını sağlar — `index` tabanlı key, listenin başına ekleme yapılınca yanlış eşleşir.

Koşullu Render Kalıpları

`&&` (koşul doğruysa küçük bir öğe ekler), üçlü operatör `? :` (iki alternatiften biri), erken dönüş (`if (...) return <...>;` — özel bir durumu geri kalan JSX'ten önce ayırır).

Boş Liste Durumu

`etkinlikler.length === 0` iken boş bir sayfa yerine anlamlı bir mesaj ("Henüz etkinlik yok.") gösterme alışkanlığı.

Ortak Hata / Tuzak

`key` eklemeyi unutmak derleme hatası değildir, kod çalışır — ama React konsola uyarı basar; `key` olmadan React öğeleri yalnızca sıralarına göre eşler, listeye yeni öğe eklendiğinde yanlış öğe durumu taşıyabilir.

» Komutlar

`npm create vite@latest -- --template react-ts`	Saf React+TypeScript Vite projesi oluşturur
`npm run dev`	HMR ile anlık geliştirme sunucusunu localhost:5173'te açar
`npx tsc -b`	JS üretmeden tüm projeyi tip hatalarına karşı denetler
`npm run build`	tsc denetiminden geçirip optimize edilmiş dist/ üretir

5 • İzleme **Nerede Kullanılır:** Tekrar kullanılan JSX parçası → bileşen · Dışarıdan gelen salt-okunur veri → `props` + `interface` · Açılış/kapanış arası içerik → `children` · Render'lar arası hatırlanan/değişen veri → `useState` · Diziden bileşen üretme → `map` + veriden gelen `key` · Boş/özel durum → erken dönüş ya da üçlü operatör